

AI-Powered Collaborative Trip Planner

PHASE 2

System & Software Design Document

Prepared as the design-phase deliverable following the Phase 1 Software Requirement Specification (SRS).

Document Type	Software Design Document (SDD)
Based On	Phase 1 — SRS v1.0
Architecture	MERN Stack (MongoDB, Express, React, Node.js)
Status	Design Finalized — Ready for Implementation (Phase 3)

Table of Contents

1. Introduction
2. Design Goals & Principles
3. System Architecture
4. Module-Wise Design
5. Database Design
6. API Design
7. Frontend Component & Folder Design
8. Key Process Flows
9. Security Design
10. Non-Functional Design Mapping
11. Deployment Architecture
12. Conclusion & Next Steps (Phase 3)

1. Introduction

1.1 Purpose of this Document

This document defines the **system and software design** of TripMate. It translates the functional and non-functional requirements captured in the Phase 1 SRS into a concrete architecture — covering system layers, module responsibilities, database schema, API contracts, frontend structure, key process flows, and deployment topology. It is the bridge between “**what the system must do**” (SRS) and “**how the system will be built**” (implementation, Phase 3).

1.2 Scope

The design covers all modules identified in the SRS: User Management, Identity Verification, Trip Planning, AI Trip Planner, Collaboration, Real-Time Chat, Map Integration, Expense Management, Emergency Support, Offline Access, Notifications, and Reviews & Ratings.

1.3 Reference Document

Phase 1 — Software Requirement Specification (SRS): “TripMate — AI Powered Collaborative Trip Planner”, defining functional requirements, non-functional requirements, stakeholders, and the recommended technology stack.

2. Design Goals & Principles

The design follows five guiding principles, each directly traceable to a non-functional requirement in the SRS:

Principle	Design Response	Traced NFR
Modularity	Feature-based modules with clear boundaries (Auth, Trips, AI, Maps, Emergency, etc.)	Maintainability
Layered Architecture	Strict separation: Presentation → Application → Data	Maintainability
Statelessness at Scale	Session store externalized to MongoDB via connect-mongo	Scalability
Security by Default	Sessions + bcrypt + Helmet + CORS + rate limiting at the API gateway level	Security
Fail-Safe Design	Centralized error handling, retries, graceful degradation to offline cache	Reliability

3. System Architecture

3.1 High-Level Architecture

TripMate follows a **three-tier architecture** built on the MERN stack, with a set of external services integrated at the application layer for AI, maps, media, and email.

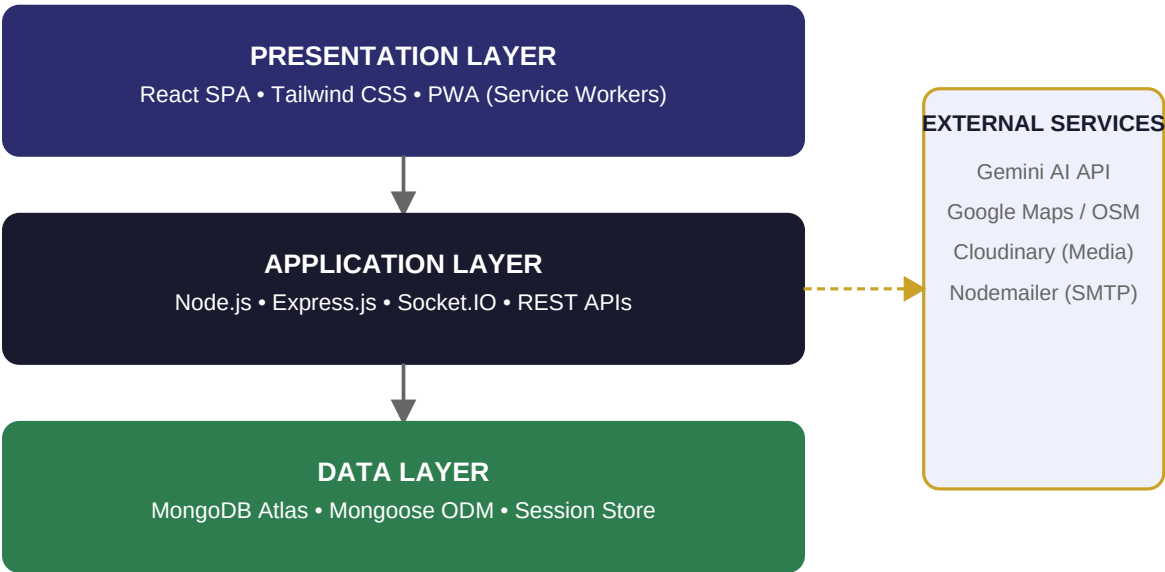


Figure 3.1 — Three-Tier System Architecture

3.2 Technology Stack

Layer	Technology
Frontend	React, React Router, Tailwind CSS, PWA (Service Workers + IndexedDB)
Backend	Node.js, Express.js, Socket.IO
Database	MongoDB Atlas (Mongoose ODM)
Authentication	Express Session, connect-mongo, bcrypt, Email OTP
AI	Gemini API
Maps	Google Maps API / OpenStreetMap
Media	Cloudinary
Notifications	Nodemailer
Deployment	Vercel (Frontend), Render (Backend), MongoDB Atlas (Database)

3.3 Architectural Style

- **MVC Pattern** on the backend — Routes → Controllers → Models, keeping business logic out of route handlers.
- **RESTful APIs** for standard CRUD operations (trips, users, expenses, reviews).

- **Event-driven communication** via Socket.IO for real-time chat and live location sharing.
 - **Service layer** isolating third-party integrations (Gemini, Maps, Cloudinary) from controllers.
-

4. Module-Wise Design

Each functional module from the SRS is mapped below to its design responsibility, primary data owned, and key interactions with other modules.

Module	Responsibility	Primary Collection	Interacts With
User Management	Registration, login, session auth, profile, password recovery	Users	Identity Verification, Notifications
Identity Verification	Document upload, verification status workflow (Unverified → Pending → Verified)	VerificationDocuments	User Management, Reviews
Trip Planning	Create / edit / delete / join / leave trips	Trips, TripMembers	AI Planner, Collaboration, Expense
AI Trip Planner	Calls Gemini API to generate itinerary, budget, packing list, weather advisory	Trips (embedded AI output)	Trip Planning, Notifications
Collaboration	Join requests, accept/reject members, group discussion	TripMembers	Trip Planning, Chat, Notifications
Real-Time Chat	Chat rooms per trip, media sharing, live location	Chats	Collaboration, Map Integration
Map Integration	Navigation, nearby services (hospitals, fuel, police, hotels)	— (external API driven)	Emergency, Trip Planning
Expense Management	Shared expenses, auto-splitting, payment history	Expenses	Trip Planning, Notifications
Emergency Module	SOS, first-aid guide, emergency contacts, hospital lookup	EmergencyResources	Map Integration, Offline Support
Offline Support	Caches critical data via Service Workers + IndexedDB	Cached subset of Trips, Emergency Resources	Emergency, Trip Planning
Notification Module	Email notifications for key lifecycle events	Notifications	User Management, Trip Planning, Expense
Reviews & Ratings	Rate travelers, write reviews, report misconduct	Reviews	User Management

5. Database Design

5.1 Collection Relationship Overview

MongoDB is document-oriented, so relationships below are logical references (ObjectId foreign keys) rather than SQL joins.

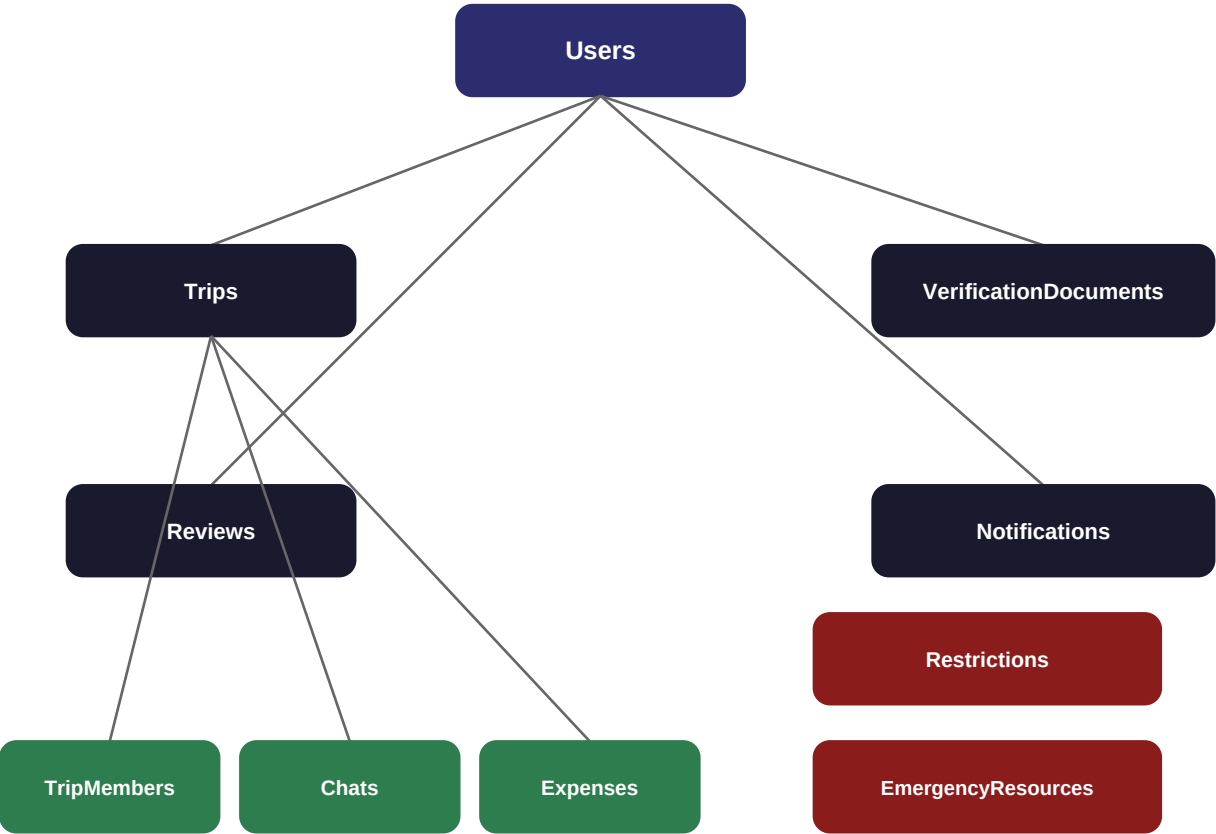


Figure 5.1 — Collection Relationship Overview

5.2 Core Schema Definitions

Users

Field	Type	Notes
name, profilePhoto	String	Basic profile
age, gender, bloodGroup	String / Number	Emergency-relevant profile data
phoneNumber, emergencyContact	String	Used by Emergency Module
medicalConditions	String	Optional, shown only in emergencies
vehicleInfo	Object	Used in trip vehicle matching
verificationStatus	Enum	Unverified / Pending / Verified
passwordHash	String	bcrypt hashed

Trips

Field	Type	Notes
destination, budget, date, duration	Mixed	Core trip metadata
vehicle, maxMembers, difficultyLevel	Mixed	Planning constraints
organizerId	ObjectId → Users	Trip owner
allitinerary	Object	Cached Gemini output (itinerary, packing list, tips)

Other Collections (Summary)

Collection	Purpose
TripMembers	Join-request state, per-trip role, membership status
Chats	Trip-scoped messages, media references, timestamps
Expenses	Shared cost entries, per-member contribution, split logic
Notifications	Queued/sent email notification records
EmergencyResources	Hospitals, ambulance, police, mechanic contact data
Restrictions	Region-specific travel restrictions (offline-cacheable)
Reviews	Traveler ratings, review text, misconduct reports
VerificationDocuments	Uploaded ID documents + verification workflow state

6. API Design

REST endpoints are grouped by module. All routes are prefixed with `/api`.

Auth

Endpoint	Description
POST <code>/auth/register</code>	Register a new user
POST <code>/auth/login</code>	Create session
POST <code>/auth/logout</code>	Destroy session
POST <code>/auth/verify-email</code>	Confirm email OTP

Trips

Endpoint	Description
POST <code>/trips</code>	Create trip
GET <code>/trips/:id</code>	Get trip details
PUT <code>/trips/:id</code>	Edit trip
DELETE <code>/trips/:id</code>	Delete trip
POST <code>/trips/:id/join</code>	Send join request

AI Planner

Endpoint	Description
POST <code>/trips/:id/ai-itinerary</code>	Generate itinerary via Gemini API

Emergency

Endpoint	Description
GET <code>/emergency/nearby</code>	Nearby hospitals, police, mechanics
POST <code>/emergency/sos</code>	Trigger SOS alert

Expenses

Endpoint	Description
POST <code>/trips/:id/expenses</code>	Add expense entry
GET <code>/trips/:id/expenses/summary</code>	Split summary

7. Frontend Component & Folder Design

7.1 Project Folder Structure

```
tripmate/
├── backend/
│   ├── config/db.js
│   ├── middleware/errorHandler.js
│   ├── models/, controllers/, routes/
│   └── server.js
└── frontend/
    ├── src/components/
    ├── src/pages/
    ├── src/services/ (API calls)
    └── src/context/ (auth, trip state)
```

7.2 Key Component Groups

Component Group	Responsibility
Auth Components	Login, Register, ProfileForm, VerificationUpload
Trip Components	TripCard, TripForm, TripDetails, MemberList
AI Components	ItineraryView, BudgetEstimator, PackingChecklist
Chat Components	ChatWindow, MessageBubble, MediaShare
Map Components	MapView, NearbyServicesPanel, RouteNavigator
Emergency Components	SOSButton, FirstAidGuide, EmergencyContactList
Shared/UI Components	Navbar, Modal, Toast, Loader, DarkModeToggle

8. Key Process Flows

8.1 User Login Flow



Figure 8.1 — Session-based Login Flow

8.2 AI Itinerary Generation Flow

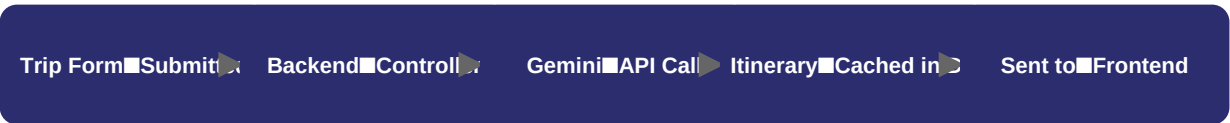


Figure 8.2 — AI Trip Planner Flow

8.3 Emergency SOS Flow



Figure 8.3 — SOS Emergency Flow

9. Security Design

Concern	Design Control
Authentication	Server-side sessions via express-session + connect-mongo (survives restarts)
Password Storage	bcrypt hashing, never stored in plaintext
Transport Security	HTTPS enforced end-to-end, secure cookies (httpOnly, secure flags)
Input Validation	Schema-level validation on every controller before DB writes
Abuse Prevention	Rate limiting on auth and AI-itinerary endpoints
HTTP Hardening	Helmet middleware for secure HTTP headers
Cross-Origin Control	CORS restricted to known frontend origins
Identity Trust	Document-based identity verification workflow reduces fake profiles

10. Non-Functional Design Mapping

NFR (from SRS)	Design Mechanism
Performance (<2s response)	SPA client-side routing, indexed MongoDB queries, cached AI responses
Scalability (10,000+ users)	Stateless app servers + externalized session store, horizontal scaling on Render
Availability (99.9%)	Managed hosting (Vercel/Render/Atlas) with automatic failover
Reliability	Centralized asyncHandler + errorHandler middleware, retry logic on external API calls
Maintainability	MVC pattern, modular routes/controllers, reusable React components
Compatibility	Responsive Tailwind layout tested across Chrome, Firefox, Edge, mobile browsers

11. Deployment Architecture

The system is deployed across managed platforms to minimize operational overhead while meeting the availability and scalability targets from the SRS.

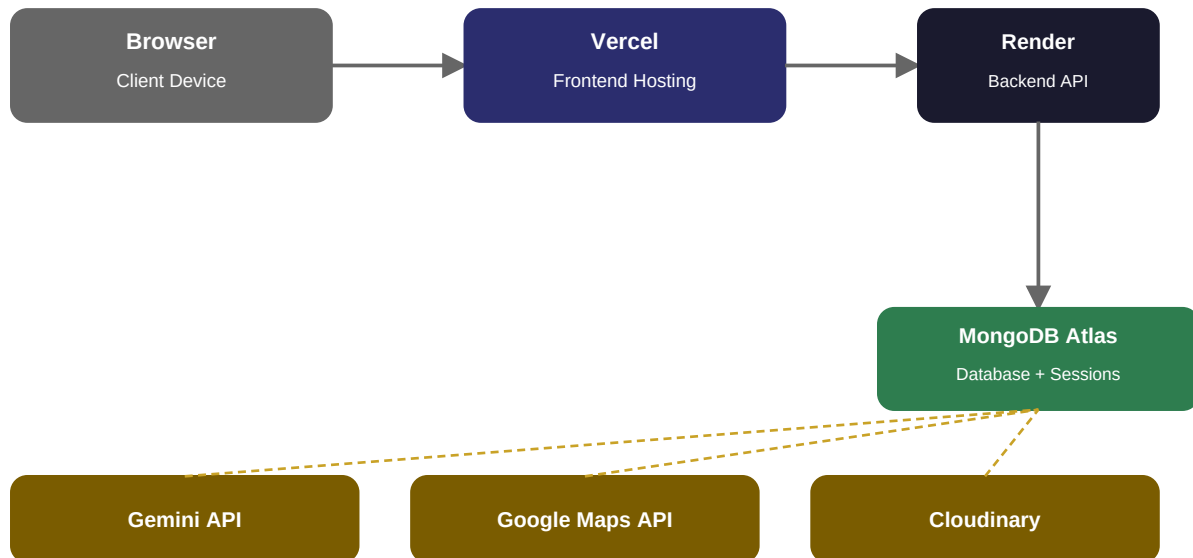


Figure 11.1 — Deployment Topology

12. Conclusion & Next Steps

This design translates every functional and non-functional requirement from the Phase 1 SRS into a concrete, buildable architecture: a three-tier MERN system, twelve well-bounded modules, a MongoDB schema, a REST + Socket.IO API surface, a component-based frontend, and a managed deployment topology.

Phase 3 will begin implementation against this design — starting with the Auth and Trip modules, followed by AI integration, real-time chat, maps, and the emergency/offline modules, in that order of dependency.

End of Phase 2 — Design Document